

1. What is the difference between a function and a procedure in PostgreSQL?

У PostgreSQL функція завжди повертає значення і може використовуватися безпосередньо в SQL-запитах. Процедура не повертає значень, викликається командою `CALL` і може самостійно керувати транзакціями(типу відкату або коміту)

2. Can a trigger be executed manually? Why or why not?

Ні, тригер не можна викликати вручну. Він є подієво-орієнтованим і автоматично спрацьовує виключно у відповідь на конкретні дії в базі даних, такі як INSERT, UPDATE або DELETE для прив'язаної таблиці

3. What are the advantages and disadvantages of storing business logic inside the database?

Переваги

Логіка централізована, продуктивність вища через відсутність зайвих мережових запитів, дані залишаються консистентними незалежно від клієнта.

Недоліки

Сервери баз даних складніше горизонтально масштабувати, а код у БД важче покривати тестами, дебажити та контролювати через Git порівняно з класичним бекендом.

Аналіз запиту (Bonus Task 3)

```
EXPLAIN ANALYZE
SELECT
    oi.order_id,
    p.product_name,
    oi.quantity,
    oi.price,
    oi.quantity * oi.price AS item_total
FROM order_items oi
JOIN products p ON oi.product_id = p.product_id
WHERE oi.order_id = 1;
```

PostgreSQL використовує сканування щоб знайти відповідні рядки в таблиці `order_items` за умовою `order_id = 1`. Після цього рушій бази даних застосовує алгоритм з'єднання щоб співставити знайдені деталі замовлення з інформацією з таблиці `products` за допомогою зовнішнього ключа `product_id`. `EXPLAIN ANALYZE` дозволяє побачити реальний час виконання операції та прогнозований час

The screenshot shows a PostgreSQL IDE interface. On the left, the 'Database Explorer' pane displays a tree view with 'postgres@localhost' and 'stores' database. The 'console' pane on the right contains the following SQL commands:

```
1 CALL create_order( p_customer_id 1);
2
3 CALL add_product_to_order( p_order_id 4, p_product_id 1, p_quantity 1);
4 CALL add_product_to_order( p_order_id 4, p_product_id 2, p_quantity 2);
5
6 SELECT * FROM orders WHERE order_id = 4;
7 SELECT * FROM order_items WHERE order_id = 4;
8 SELECT product_id, product_name, stock_quantity FROM products WHERE product_id IN (1, 2);
9 SELECT * FROM order_log WHERE order_id = 4;
```

The 'Output' pane at the bottom shows the results of the last command, displaying a table with columns: log_id, order_id, customer_id, action, and log_date. The first row shows: 1, 4, 1, Order Created, 2026-06-28 08:42:47.894884.

The screenshot shows the same PostgreSQL IDE interface. The 'console' pane now contains the following SQL query:

```
7 oi.quantity * oi.price AS item_total
8 FROM order_items oi
9 JOIN products p ON oi.product_id = p.product_id
10 WHERE oi.order_id = 1;
```

The 'Output' pane shows the 'QUERY PLAN' for this query. The plan details are as follows:

```
1 Hash Join (cost=27.09..41.32 rows=7 width=274) (actual time=0.034..0.036 rows=2 loops=1)
2 Hash Cond: (p.product_id = oi.product_id)
3 -> Seq Scan on products p (cost=0.00..13.00 rows=300 width=222) (actual time=0.013..0.014 rows=5 loops=1)
4 -> Hash (cost=27.00..27.00 rows=7 width=28) (actual time=0.010..0.010 rows=2 loops=1)
5 Buckets: 1024 Batches: 1 Memory Usage: 9kB
6 -> Seq Scan on order_items oi (cost=0.00..27.00 rows=7 width=28) (actual time=0.006..0.006 rows=2 loops=1)
7 Filter: (order_id = 1)
8 Rows Removed by Filter: 5
9 Planning Time: 0.201 ms
10 Execution Time: 0.060 ms
```